

Monte Carlo Simulation of BGM

19.1 Why simulation, and why it's hard

A calibrated BGM model earns its keep pricing products a closed-form formula cannot reach — Bermudan swaptions, spread options, CMS products, anything genuinely path-dependent or dependent on more than one point of the curve jointly. Chapter 13's drift formula makes this possible in principle: it specifies exactly how every forward rate must evolve, jointly, under one common measure. Making it work in practice means discretising that continuous-time SDE system into finite time steps, and Chapter 13's own worked example already showed why this is not routine: the drift at each step depends on the levels of *every* other rate, evaluated at that same instant — a fully state-dependent, correlation-weighted quantity that cannot simply be looked up in advance.

19.2 The naive approach, and why it's not good enough

The most direct discretisation is a lognormal Euler scheme: over a small step Δt , approximate

$$L_i(t + \Delta t) \approx L_i(t) \exp \left[\left(\mu_i(t) - \frac{1}{2} \sigma_i(t)^2 \right) \Delta t + \sigma_i(t) \sqrt{\Delta t} Z_i \right]$$

where $\mu_i(t)$ is Chapter 13's drift, evaluated using the *known* rate levels at the start of the step, and Z_i is a (correlated, via ρ_{ij}) standard normal draw. This is simple and exactly what a first implementation naturally reaches for — and it has a specific, well-documented weakness.

PITFALL

Evaluating the drift $\mu_i(t)$ using only the rate levels at the *start* of the step, then holding it fixed across the whole step, is a first-order (Euler) approximation to a quantity that is itself changing throughout the step — and because BGM's drift is fairly sensitive to rate levels (Chapter 13's γ_j term grows with L_j), this approximation error compounds specifically in high-volatility or long-time-step regimes, producing simulated forward rates whose distribution is measurably different from what the true, continuous-time model implies. In practice this shows up as an Euler-simulated model that fails to correctly reprice its own calibration instruments (the very caplets and swaptions used to fit it) when those prices are recomputed via Monte Carlo rather than the closed-form formulas of Chapters 6 and 7 — a serious, credibility-damaging inconsistency for a production system, since it means *the simulator itself* doesn't agree with the calibration that fed it.

19.3 The predictor-corrector fix

The standard, and by a wide margin most widely used, remedy is the **predictor-corrector scheme** (Hunter, Jäckel and Joshi, and closely related approaches).

Key result: the predictor-corrector scheme

1. **Predict.** Compute a first-pass estimate $\hat{L}_i(t + \Delta t)$ using the standard Euler step above, with the drift $\mu_i(t)$ evaluated at the *known* rate levels at time t .
2. **Correct.** Recompute the drift a second time, $\hat{\mu}_i(t + \Delta t)$, this time using the *predicted* rate levels $\hat{L}_j(t + \Delta t)$ from step 1.
3. **Average and finalise.** Take the arithmetic mean of the two drift estimates, $\bar{\mu}_i = \frac{1}{2}(\mu_i(t) + \hat{\mu}_i(t + \Delta t))$, and use *this* averaged drift in the final Euler step to produce $L_i(t + \Delta t)$, reusing the *same* random draws Z_i from step 1.

This is structurally identical to a predictor-corrector (or Heun-type/trapezoidal) scheme for an ordinary differential equation: using the average of the drift at the start and (an estimate of) the end of the step is a second-order-accurate approximation to the drift's true path across the step, at roughly double the computational cost of plain Euler (two drift evaluations per step instead of one) but a materially better fit to the model's own closed-form prices.

Worked example 19.1 — one predictor-corrector step, by hand.

Reuse the two-rate setup from Worked Example 13.1: $\tau = 0.5$, $L_1(t) = 4.20\%$, $L_2(t) = 4.35\%$, $\sigma_1 = 22\%$, $\sigma_2 = 24\%$, $\rho_{12} = 0.85$, terminal measure, single time step $\Delta t = 0.5$. Suppose a single correlated normal draw gives $Z_1 = 0.40$ (holding L_2 's own drift/diffusion fixed for this illustration, to isolate the effect on L_1 alone).

Step 1 — predictor drift. From Worked Example 13.1, the $j = 2$ drift contribution to L_1 at time t was $\mu_1(t) \approx -4.01 \times 10^{-5}$ (per unit time; treat this as the full drift for the illustration).

Step 2 — predictor Euler step.

$$\hat{L}_1(t + \Delta t) = L_1(t) \exp \left[\left(\mu_1(t) - \frac{1}{2} \sigma_1^2 \right) \Delta t + \sigma_1 \sqrt{\Delta t} Z_1 \right]$$

$$= 0.0420 \times \exp \left[(-0.0000401 - 0.0242) \times 0.5 + 0.22 \times 0.7071 \times 0.40 \right]$$

$$= 0.0420 \times \exp[-0.01212 + 0.06222] = 0.0420 \times \exp[0.05010] = 0.0420 \times 1.05138 = 0.04416$$

Step 3 — corrector drift. Recompute the $j = 2$ drift term using the *predicted* $\hat{L}_1 = 4.416\%$ in place of L_1 (with L_2 , γ_2 held fixed for this illustration): since the drift is linear in L_1 itself (it multiplies $\sigma_1 L_1$), the corrected drift scales by $\hat{L}_1/L_1 = 4.416/4.20 = 1.0514$:

$$\hat{\mu}_1 \approx -4.01 \times 10^{-5} \times 1.0514 = -4.216 \times 10^{-5}$$

Step 4 — average and finalise.

$$\bar{\mu}_1 = \frac{1}{2}(-4.01 \times 10^{-5} - 4.216 \times 10^{-5}) = -4.113 \times 10^{-5}$$

$$L_1(t + \Delta t) = 0.0420 \times \exp \left[(-0.0000411 - 0.0242) \times 0.5 + 0.22 \times 0.7071 \times 0.40 \right]$$

$$= 0.0420 \times \exp[-0.01212 + 0.06222] = 0.04416$$

Sanity check. With only two rates and a drift this small relative to the diffusion term, the predictor and corrector steps barely differ here (4.01 vs. 4.22×10^{-5}) — the four-decimal-place final answer is unchanged from the predictor alone at this precision. This is expected and appropriate: predictor-corrector's benefit grows with the number of

correlated rates and the size of the time step, and a minimal two-rate, half-year-step illustration is exactly the regime where the correction is smallest.

Interpretation. In a real 120-rate, long-time-step simulation, the drift terms compound across many more j indices and the gap between predictor-only and corrected drift becomes large enough to matter for pricing accuracy — which is precisely why production implementations do not skip the correction step as an optimization, however tempting the doubled computational cost makes it look.

19.4 Reducing the computational burden

Two further techniques, both already introduced conceptually earlier in the book, are essentially mandatory for a production-speed BGM Monte Carlo engine.

ON THE DESK

Factor reduction (Chapter 14) is applied directly at the simulation level: instead of drawing N correlated normals per time step (one per forward rate, requiring a full $N \times N$ correlation structure to be applied via Cholesky decomposition at every step), draw only r independent normals ($r = 3\text{--}5$ typically) and map them to each rate's own driving noise via the rank-reduced loading matrix. This is frequently the single largest speed-up available in a BGM Monte Carlo engine, precisely because both the random-number generation cost and the per-step correlation application cost scale with the number of factors, not the number of rates.

Variance reduction techniques — antithetic variates (pairing each path with its mirror-image, $-Z$ in place of Z) and control variates (using the known closed-form caplet/swaption price as a control for a related, harder-to-price exotic) — are routine and essentially free relative to their benefit, often cutting the number of paths needed for a given target accuracy by a factor of several times.

PITFALL

A subtler numerical issue specific to BGM, related to but distinct from the drift-approximation error above, is **long-stepping**: for computational speed, many production implementations take one large time step per tenor period (e.g. one step per 3-month period, rather than many small sub-steps within it) rather than a fine time grid. This is usually acceptable for the drift approximation itself when predictor-corrector is used, but it can materially misprice any payoff that depends on the rate's behaviour *within* a period rather than just at its endpoints — exactly the in-arrears, compounded-RFR caplets built in Chapter 3 and 5, whose entire payoff structure is defined by what happens *inside* the accrual period. A long-stepped simulation that is perfectly adequate for a standard Bermudan swaption can silently misprice an in-arrears convexity adjustment for exactly this reason, and a production system needs to know which of its products require the finer grid.

Chapter benchmark

Before moving on, you should be able to, without notes:

1. Explain why a plain Euler scheme, evaluating BGM's drift only at the start of each time step, produces a biased simulation.

2. Describe the three steps of the predictor-corrector scheme: predict, correct, average.
3. Explain what factor reduction saves at the simulation level, and why long-stepping can misprice in-arrears payoffs specifically.